

CS767 Assignment 2 — Data Analysis Agent

Student: Dongqing Yang

ID: 291140653

GitHub: https://github.com/yangdongqing214/compsci767_assignment2

.env: `OPENAI_API_KEY=sk-proj-`

`055EHEot_rqqEUCqOhlEjBAE0jIbw_R9mbhG6flmefEcauSdCwMHH7JXmO6fOyNQiaAhHSUc
6MT3BlbkFJGI4zt4vyP3efSLqSLXkc7wdioM42T7F_GsKSI6QjDG34yQhU3PoH6txldotV2Ci9iy3
MUxxYoA`

`OPENAI_MODEL=gpt-4o-mini`

Demo Video: https://drive.google.com/file/d/1OC8XZ6micZoxGgzhwfMAQEKF-NoU0Wd/view?usp=drive_link

Repository

https://github.com/yangdongqing214/compsci767_assignment2

Architecture

The agent follows a **perceive** → **decide** → **act** → **reflect** loop:

1. **Perceive:** Load CSV; compute profile (columns, types, missing counts).
2. **Decide:** Planner produces a JSON list of steps (LLM if OPENAI_API_KEY set, else heuristic rules).
3. **Act:** Each step runs a **builtin tool** (statistics, plots, groupby) or **sandboxed pandas code** validated by safety.py.
4. **Reflect:** Results stored in AgentMemory; final markdown report written to output/.

User goal + CSV

↓

[DataAnalysisAgent]

↓

profile → plan → execute steps → memory + report.md + PNG artifacts

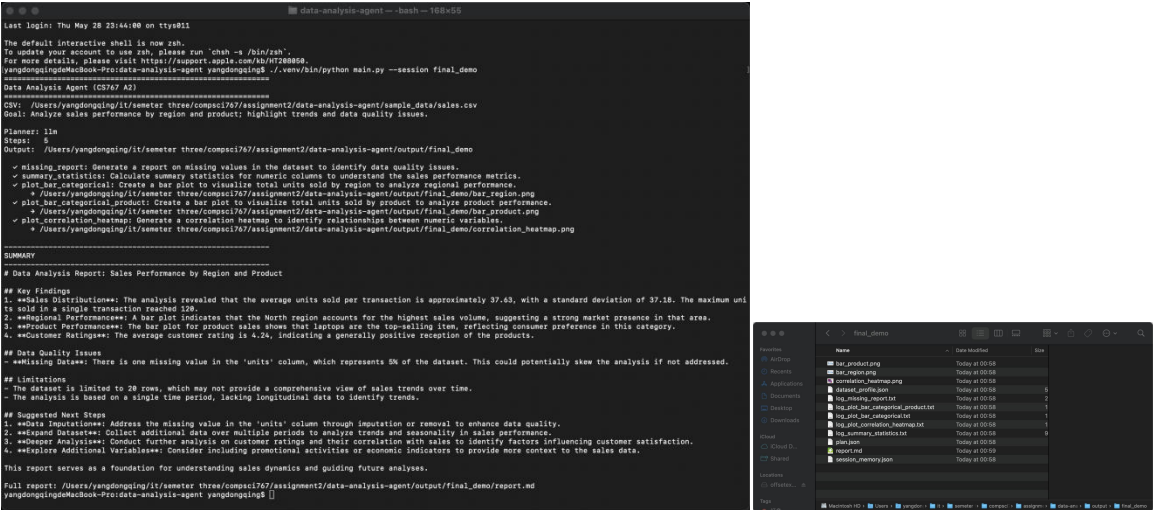
Design choices

Tools vs. free-form code: Most steps use reliable builtins; optional code step for goal-specific metrics (e.g. revenue = units × price).

Safety: AST scan blocks os, subprocess, open, network imports before exec.

Reproducibility: Heuristic mode requires no API; graders can run python main.py immediately.

Screenshot 1,2 — CLI run, Generated artifacts



- 1.Completes 5–6 steps and prints a short summary.
- 2.Histogram, correlation heatmap, and bar charts produced automatically.

How it works (short)

The user supplies a **goal** and **CSV**. The agent does not hard-code one analysis pipeline—it **selects steps from the dataset profile** and user goal, executes them, and aggregates results. Failed steps are logged without stopping the whole run (robust act loop). **For limitations**, single-pass planning — no observe→replan if a step fails. Memory logs outcomes but does not feed back into replanning within one run. Trade-off: builtin tools improve reliability over fully LLM-generated code.